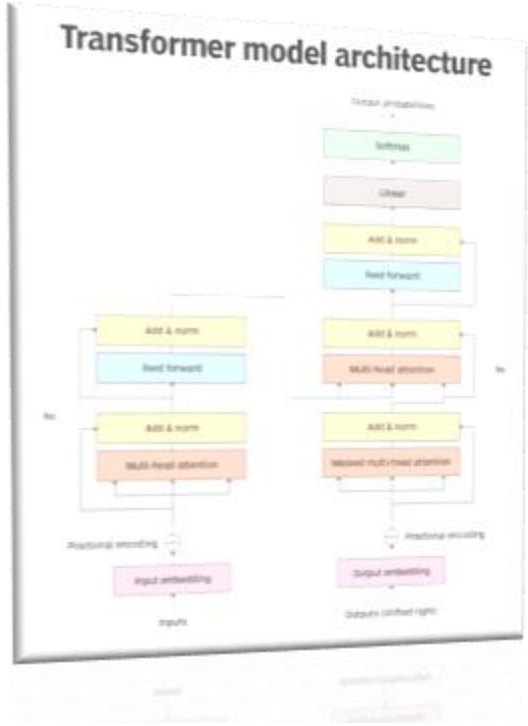


Transformers

ChEBI Molecular Classification

Multi-label prediction of 500 chemical classes from SMILES

🏆 **Ensemble F1 Micro: 0.884 | F1 Macro: 0.764**

Hybrid GNN | **Deep MLP** | **Ensemble**

Hackathon 2026

Problem & Pipeline

~40K
Training Molecules

500
Binary Classes

11K
Test Molecules

Zadanie

Input: SMILES + 7 cech atomowych

Output: Wektor binarny 500 klas

Metryki: F1 Micro & F1 Macro

Dane: ChEBI – ontologia chemiczna

Hierarchia: korekcja obonet (DAG klas)

Pipeline

① SMILES → RDKit + sanitize

② AtomPairFingerprint (skfp)

③ Graf molekularny (7 cech atomu)

④ Ontologia ChEBI → DAG (obonet)

⑤ Trenowanie GNN + MLP → Ensemble

Architektury Modeli & Ensemble

waga 60%

Hybrid GNN

ChemicalHybridNet

- GCNConv: 7 → 64 → 128 dim
- GlobalAttention pooling
- Concat graf + fingerprint
- FC: (128+FP) → 512 → 500

0.6×GNN + 0.4×MLP

waga 40%

Deep MLP

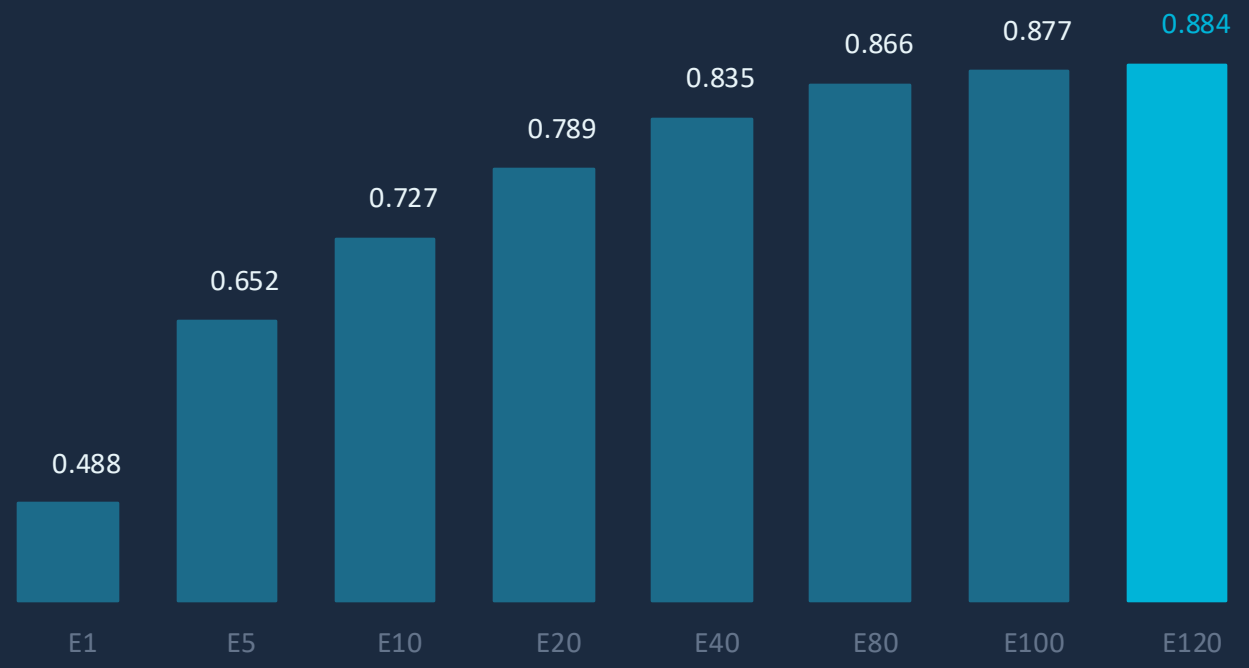
FingerprintMLP

- Input: AtomPair FP
- 1024 + BatchNorm + Dropout(0.4)
- 512 + BatchNorm + Dropout(0.3)
- Wyjście: 500 klas

Optymalizacja progów per-klasa • Korekcja hierarchiczna (apply_hierarchy) • 120 epok • ReduceLROnPlateau

Wyniki & Wnioski

Krzywa uczenia — Ensemble F1 Micro



Kluczowe wnioski

- Ensemble (GNN+MLP) pobił oba modele osobno — F1 Micro 0.884
- GlobalAttention > global_mean_pool — lepsza agregacja grafu
- BatchNorm + głębszy MLP znacznie poprawił Macro F1 (+0.21)
- Korekcja hierarchii (obonet) poprawia spójność ontologiczną

CONTEXT ENGINEERING

The Architect Strategy

3-Phase Context Construction Pipeline

Extraction → Compression → Assembly

High-precision context, low token overhead

Deterministic · Zero ML / Embeddings

Optimized for Mellum, Codestral & Qwen2.5

Code-first LLM targets

PHASE 1

Extraction — Knowledge Graph

Phase 1: Extract | Phase 2: Clean | Phase 3: Assemble

knowledge_graph_builder.py

```
import ast
class KnowledgeGraphBuilder:
  def scan_repo(self, path):
    # Recursive .py scanner
    for file in rglob('*.py'):
      tree = ast.parse(src)
      self.extract(tree)

  def extract(self, tree):
    # Classes, functions,
    # signatures, docstrings
```

Static Analysis

ast.parse + KnowledgeGraphBuilder

Recursive Repo Scan

All .py files, depth-first traversal

AST Symbol Detection

Classes, functions & full signatures

Rich Metadata

Docstrings, line numbers, type annotations

PHASE 2

Cleaning & Compression

Phase 1: Extract | **Phase 2: Clean** | Phase 3: Assemble

Token Savings

Massive reduction without losing structure

context_architect.py

Smart code stripping engine — purpose-built for context construction

Comment Removal

Strips comments & excess whitespace — preserve logic, drop noise

Signature-Only Mode

Headers & first-line docstrings only for distant dependencies

Token Budget Tracking

Live budget counter — cuts until target met, keeps structure intact

PHASE 3

Context Assembly

Phase 1: Extract | Phase 2: Clean | **Phase 3: Assemble**

Priority Stack

① **Local Context**
~50 lines before / 20 after cursor

② **Imported Symbols**
Resolved via Knowledge Graph

③ **Inheritance Chain**
Parent class definitions

④ **Secondary Fill**
Remaining budget (no tests)

Priority-based Heuristics

Deterministic ranking — no guessing

Knowledge Graph Lookup

Symbol resolution from Phase 1

Context Prioritization Logic

Layered — local → imported → parents

Inheritance-Aware

Full parent class context included

ADVANCED

Matching & Pattern Recognition

16k Token Budget

← 8k Primary Logic → ← 8k Secondary →

- ✅ Pattern matching on class names
- 🔁 Sibling identification
- 🧪 Tests excluded from secondary fill
- 📐 Repeating implementation patterns

🔎 Class Name Pattern Matching

Identifies 'sibling' classes sharing naming conventions — pulls all relevant variants automatically

📦 Secondary Context Fill

Fills remaining token budget with related symbols — tests are explicitly excluded to reduce noise

🔄 Repeating Pattern Recognition

Detects recurring implementation patterns across the codebase — boosts completion accuracy

Predictive Grid Management for Heat Pump Networks

Euros Energy Hackathon — Task 3

600	2.9M	~0.0075	58
devices	samples/hour	Holdout MAE	features

Problem: Extrapolation to Summer

Dataset Split

Train *7 mo. | winter* ❄️

Oct 2024 – Apr 2025

Val *2 mo. | summer* 🌱

May – Jun 2025

Test *4 mo. | summer* ☀️

Jul – Oct 2025

⚠️ Distribution Shift

Feature	Train	Val
imgw_t_mean_C	2.7°C	15.6°C
heating_degree_days	15.3	2.4
daylight_hours	9.9h	16.0h

The model never saw summer temperatures!

💡 Our Solution: Train Hourly, Aggregate Monthly

❌ Old Approach

Hourly data

▼ aggregate to months

Monthly features

(imgw, HDD, season...)

▼ model

Monthly prediction

MAE 0.0225 → 0.0211 on leaderboard

VS

✅ New Approach

Hourly data

▼ physical features per hour

t1–t13, deltas, time...

▼ model → x2 per hour

▼ mean(x2) per month

Monthly prediction ✓

Holdout MAE ~0.0075 🎯

🛠️ Feature Engineering — 58 Features in 4 Categories

🌡️ Physical

- t1–t13 (sensor temperatures)
- load_source_delta
- load_hex_delta
- indoor_outdoor delta

→ Stable across seasons

🕐 Temporal

- hour_sin/cos, month_sin/cos
- dow_sin/cos, doy_sin/cos
- is_peak_hour, is_night
- is_daylight

→ Continuous cycle encoding

📍 Geographic

- lat_norm, lon_norm
- DSO region (Enea/PGE/...)
- nearest_centroid_km
- imgw_t_mean_C (IMGW norms)

→ Regional differences in Poland

📊 Per-Device

- device_x2_mean_train
- device_x2_warm/cold
- dev_t1_slope (temp curve)
- dev_linear_pred

→ Key for summer extrapolation

Model: Residual MLP + Optuna

Best Hyperparameters — Optuna TPE (5 trials, A100 40GB)

hidden_dim	256	n_blocks	4	dropout	0.204
learning_rate	5.39e-4	batch_size	2048	optimizer	AdamW

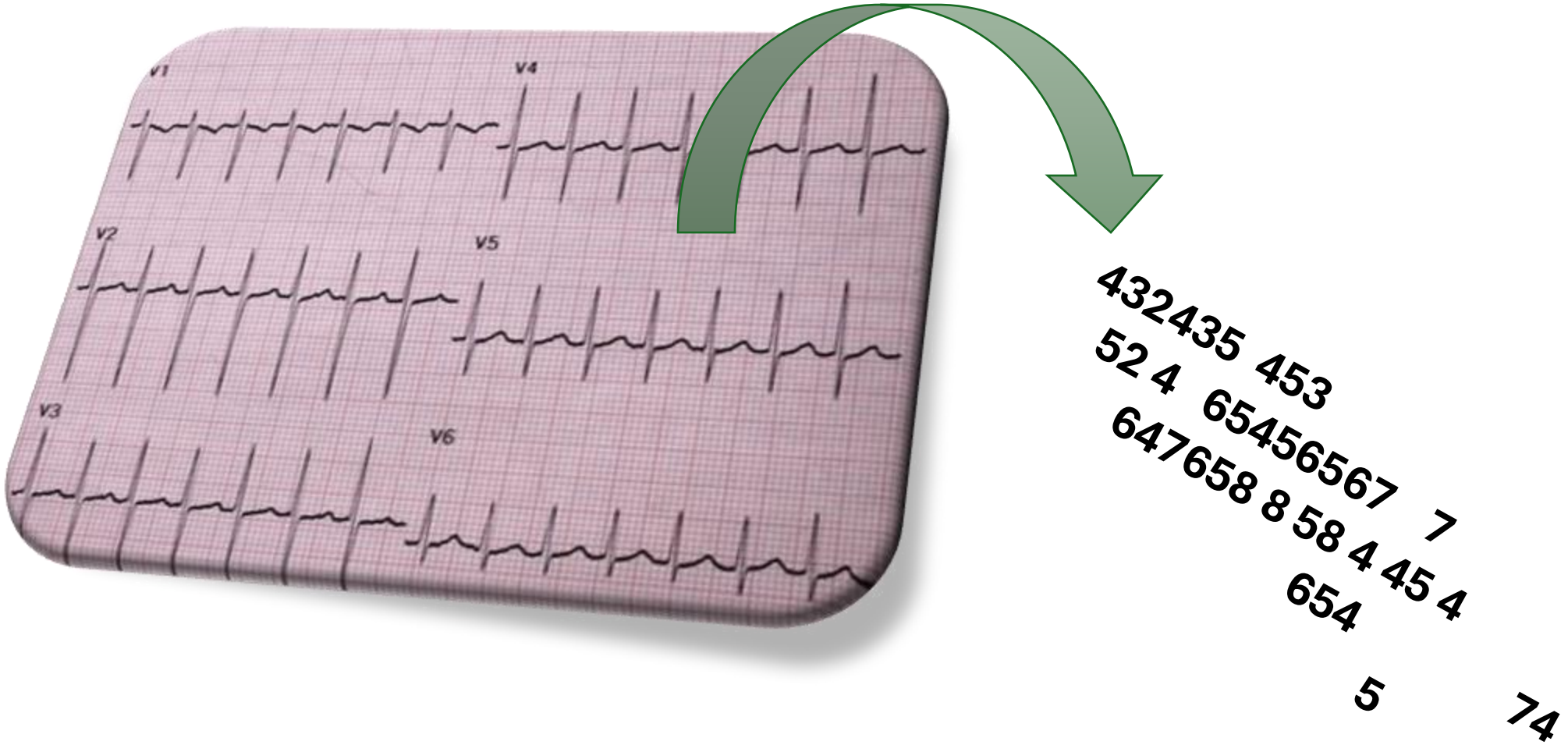
Results & Summary

Leaderboard / Holdout MAE (lower is better)

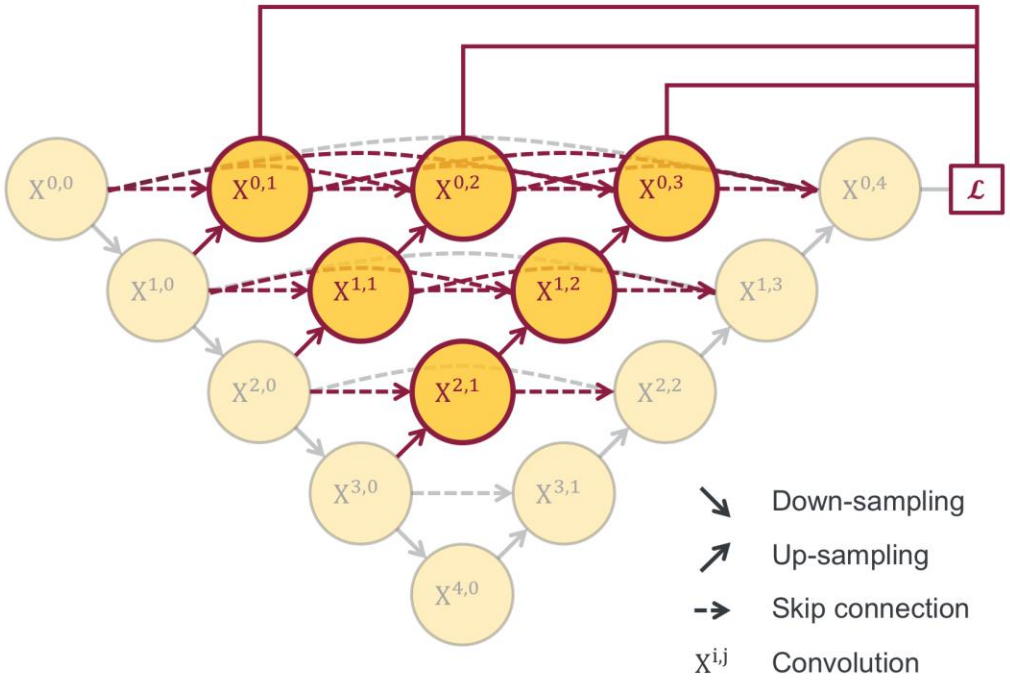
Key Takeaways

- Training on hourly data gave 3× lower MAE than monthly aggregates
- Physical features (t4–t8) are stable across seasons — model generalises to summer
- Per-device temperature slope (dev_linear_pred) is the key extrapolation feature
- Residual MLP + AMP on A100 — 2.9M samples processed in ~40s/epoch

Task4: ECG Digitization

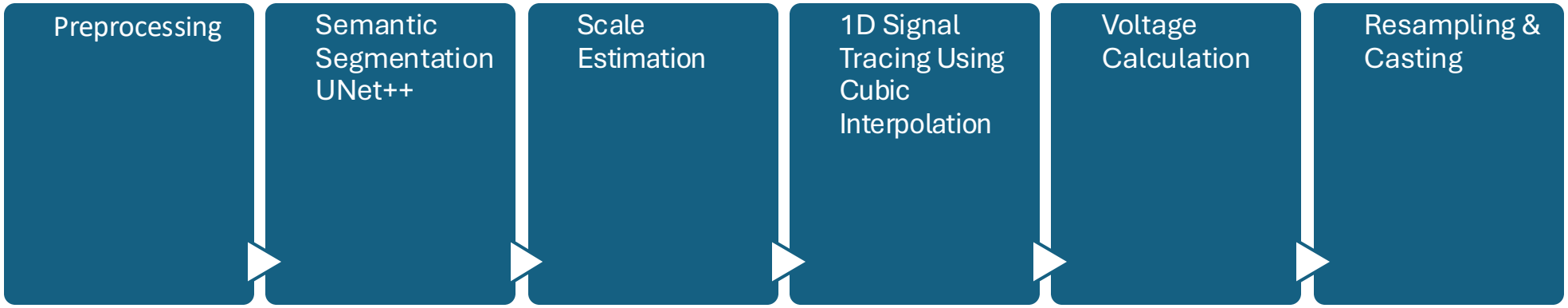


UNet++ - A nested UNet Architecture



UNetpp is a nested U-Net architecture designed to reduce the semantic gap between encoder and decoder feature maps through a series of dense skip connections and optional deep supervision.

Pipeline



The pipeline implements a comprehensive end-to-end workflow that integrates automated image deskewing and multi-channel UNet++ segmentation with precise physical scale calibration to transform raw ECG photos into standardized digital signals.